18. Android, MySQL, PHP, & JSON 2 | Setting u XAMPP Server and MySQL

Posted By trav on May 28, 2013 | 0 comments

This entry is part 18 of 22 in the series Android-Intermediate

Setting up an Apache Server, MySQL Database, and a PHP Environment

Welcome to the second part of this Android connectivity with a remote server mini series! We will be installing XAMPP, which allows us to test our live php code on a local apache server. If this makes no sense, don't worry, you'll be a coding pro in no time. ****If you already have a hosting plan and your own domain, skip to the next tutorial. This tutorial is only for those that don't have access to a server.****

Follow the Remote Databases for Android Series!

- Part 1: Android Remote Databases and Servers Overview
- **Part 2: Local Xampp Server and MySQL Database**
- Part 3: Working with a Remote Server and MySQL
- Part 4: Creating Login and Registration PHP Scripts
- Part 5: Developing the Android Application
- Part 6: JSON Parsing and Android ListView Design

XAMPP Apache Server

and MySQL Database Video Tutorial

Part A: Setting up XXAMP's Apache Server

Part B: Creating MySQL

Database and A Privileged User

Part C: Connecting to MySQL with a PHP Config File

Setting up an Apache Server and MySQL Database with XAMPP

Step 1: Download and Install XAMPP

In order for us to test interacting with a remote database from an Android application, we will need to make our computer pretend like it is a server. Our pretend apache server will store our MySQL database and our web service containing PHP scripts, meaning that we will be able to test our application as if it was live. The problem is, is that we will need to download and install some software in order to make this work. We will be using XAMPP as our apache server. Download XAMPP for your operating system (it may take a while) and make sure it includes MySQL. It has been a while since I've installed XAMPP on my computer, but pretty sure there's nothing special, so just install it into the default location.

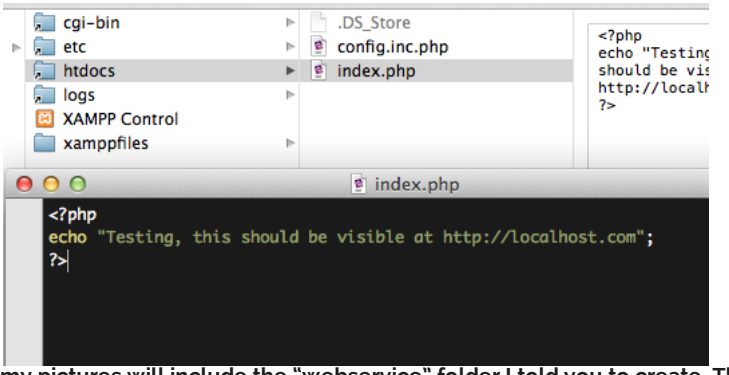
After you have everything installed, I want to do a quick test to make sure our XAMPP server is running properly. There are two important details for working with XAMPP, one is that our files should be held within the htdocs folder, and the second is that we need to turn our server on via the XAMPP control panel. Locate your where XAMPP was installed on your system (mac: /Applications/XAMPP/), and open up the XAMPP Control application. Once that it open just running the Apache server and the MySQL server. If you're on mac, you may have to type in your mac password:
xampp-start
(If you're using a different version of XAMPP, it may look a little bit different, but it will still be pretty similar.)

Step 2: Testing XAMPPs Server on LocalHost

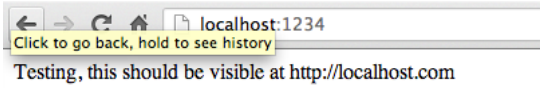
Now, we are running our apache server and MySQL. Let's test out how we use it. Back in your XAMPP directory, there should be a folder called htdocs, this will be your root folder. Create a folder called "webservice" and within that folder create two PHP files and save them here. Call the two php scripts "index.php" and "config.inc.php" (you can use notepad, or any text editor to save these files). We will modify the "config.inc.php" later, but let's create a simple script to test our Apache server.

index.php

```
1 <?php
2 echo "Testing, this should be visible at http://localhost";
3 ?>
```



****Note that none of my pictures will include the "webservice" folder I told you to create. That is because I complete forgot about doing this, so please ignore my mistake**
Save this file, and then type "http://localhost" (or "http://localhost:80") in the browser of your choice and as long as your apache server is running, you should see something like the image below:



×

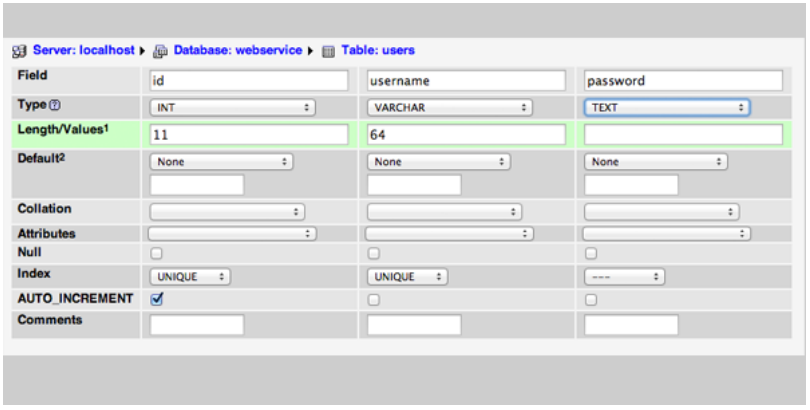
Having problems?

If yours isn't working there could be a couple of things going on. For example, other application such as skype, spotify, etc could be using the same port as the default port XAMPP is using. Also, there is a built in apache server for the mac OS that could be interfering as well. All we need to do is change our xampp server port to something besides the default (http://localhost:80). If you look at the image I posted above, you may have noticed I changed my port to 1234. This is a quick fix, just open up the XAMPP/xamppfiles/etc/httpd.conf file and the second line of uncommented code should be "Listen 80", that means it is listening for port 80 by default, let's change this to "Listen 1234", save the file, and go back to our browser and test out "http://localhost:1234". Hopefully, that will work.

Step 3: Working with XAMPP's MySQL

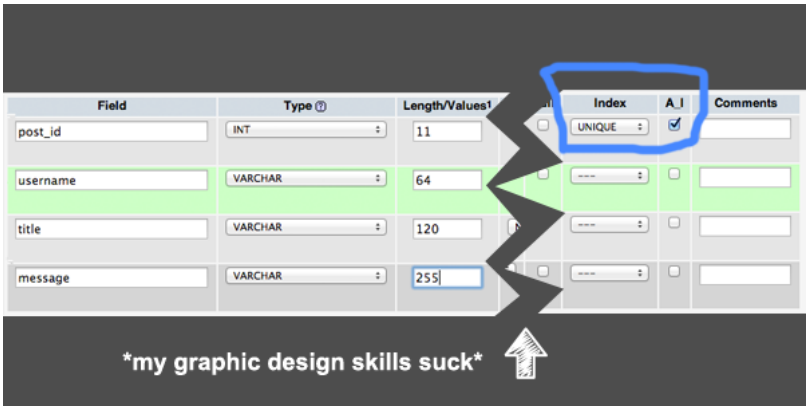
Cool, now that we know that our XAMPP server is working, let's setup the MySQL database for our application. Make sure MySQL is running on our XAMPP Control Panel and then go to http://localhost:1234/phpmyadmin (I changed my port to 1234, your's may be 80).

On the phpmyadmin page there should be a place to create a new database. Let's create a new database named "webservice". Just type it in and click create. It should have created the database successfully, and if so, in the left sidebar "webservice" should be listed within the list of databases. Click on it and now we need to add two tables, 'users' and 'comments'. The users' table will have three fields: id, username, and password. Create a new table called users and say that it has three fields, and fill in the field attributes as listed below and press save:



I know that I'm not explaining a whole lot about what we are doing right now, but I will cover this more in the next tutorial.

Let's create a new table called 'comments'. The 'comments' table will have 3 fields: username, title, and message:

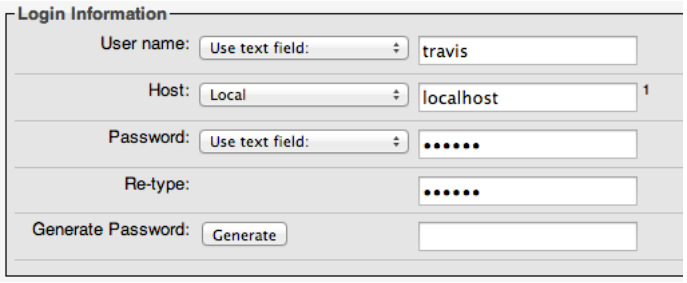


Click save and we are done setting up our database!

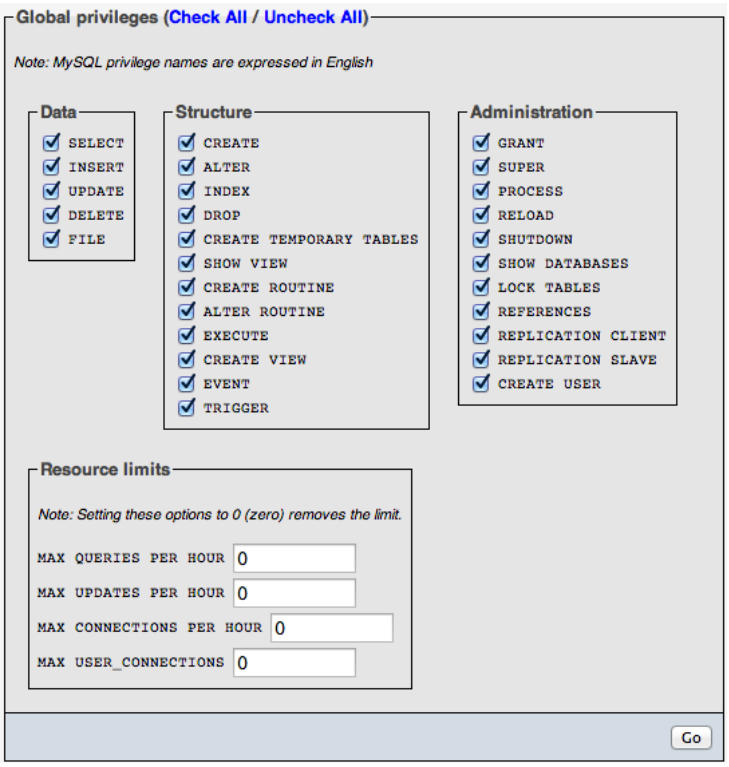
Step 4: Adding a MySQL Privileged User

Now, we have our database setup we want to be able to add data to it. In order for this to happen we need to create a user that has the privilege make such changes to our database. I will go into greater detail about this in the next tutorial as well. By default, XAMPP has already created a privileged user for use, his name is "root" and he doesn't have a password, but he isn't much fun, so let's a new user.

Go back to the phpmyadmin homepage (http://localhost:1234/phpmyadmin) and click on the "Privileges" tab. Right under the list of current user, there should be an option to "Add a new User". Click on that, and fill out the following info, make sure you remember this for later:



In a live version, you would want the user's password to be very secure, because if someone hacks the password, they could manipulate the data as they please. However, in this example, my password is bacon1. You will want to remember your username and password for later, so you may want to write it down somewhere. You will also want to give this user all privileges:



Click "GO" and we have created our new user!

Step 5: Setting Up Our PHP Config File.

Now, we have all of the information that we need to create our PHP based web service that communicates with our Android app and our MySQL Database. Our web service will have to send the database and privileged user credentials every time that we want add to, select from, update, or delete anything from our MySQL database. This would be a tedious process if we had to write this in every php script, so instead we will have one php script that holds our database info and it will automatically try to connect every time we include the file. We named this commonly used php script 'config.inc.php', so let's edit this file with our credentials:

config.inc.php

This modified script is from an excellent commented php file on building a secure login system, and I just wanted to give credit where credit is due. login.http://forums.devshed.com/php-faqs-and-stickies-167/how-to-program-a-basic-but-secure-login-system-using-891201.html

```
1  <?php
2
3      // These variables define the connection information for your MySQL database
4      $username = "travis";
5      $password = "bacon1";
6      $host = "localhost";
7      $dbname = "webservice";
8
9  // UTF-8 is a character encoding scheme that allows you to conveniently store
10     // a wide varienty of special characters, like ¢ or €, in your database.
11     // By passing the following $options array to the database connection code we
12     // are telling the MySQL server that we want to communicate with it using UTF-8
13     // See Wikipedia for more information on UTF-8:
14     // http://en.wikipedia.org/wiki/UTF-8
15     $options = array(PDO::MYSQL_ATTR_INIT_COMMAND => 'SET NAMES utf8');
16
17     // A try/catch statement is a common method of error handling in object oriented
   code.
18     // First, PHP executes the code within the try block.  If at any time it
   encounters an
19     // error while executing that code, it stops immediately and jumps down to the
20     // catch block.  For more detailed information on exceptions and try/catch
   blocks:
21     // http://us2.php.net/manual/en/language.exceptions.php
22     try
23     {
24         // This statement opens a connection to your database using the PDO library
25         // PDO is designed to provide a flexible interface between PHP and many
26         // different types of database servers.  For more information on PDO:
27         // http://us2.php.net/manual/en/class.pdo.php
28         $db = new PDO("mysql:host={$host};dbname={$dbname};charset=utf8", $username,
   $password, $options);
29     }
30     catch(PDOException $ex)
```

```
    {
        // If an error occurs while opening a connection to your database, it will
        // be trapped here.  The script will output an error and stop executing.
        // Note: On a production website, you should not output $ex->getMessage().
        // It may provide an attacker with helpful information about your code
        // (like your database username and password).
        die("Failed to connect to the database: " . $ex->getMessage());
    }

    // This statement configures PDO to throw an exception when it encounters
    // an error.  This allows us to use try/catch blocks to trap database errors.
    $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // This statement configures PDO to return database rows from your database using
an associative
    // array.  This means the array will have string indexes, where the string value
    // represents the name of the column in your database.
    $db->setAttribute(PDO::ATTR_DEFAULT_FETCH_MODE, PDO::FETCH_ASSOC);

    // This block of code is used to undo magic quotes.  Magic quotes are a terrible
    // feature that was removed from PHP as of PHP 5.4.  However, older installations
    // of PHP may still have magic quotes enabled and this code is necessary to
    // prevent them from causing problems.  For more information on magic quotes:
    // http://php.net/manual/en/security.magicquotes.php
    if(function_exists('get_magic_quotes_gpc') && get_magic_quotes_gpc())
    {
        function undo_magic_quotes_gpc(&$array)
        {
            foreach($array as &$value)
            {
                if(is_array($value))
                {
                    undo_magic_quotes_gpc($value);
                }
                else
                {
                    $value = stripslashes($value);
                }
            }
        }

        undo_magic_quotes_gpc($_POST);
        undo_magic_quotes_gpc($_GET);
        undo_magic_quotes_gpc($_COOKIE);
    }

    // This tells the web browser that your content is encoded using UTF-8
    // and that it should submit content back to you using UTF-8
    header('Content-Type: text/html; charset=utf-8');

    // This initializes a session.  Sessions are used to store information about
    // a visitor from one web page visit to the next.  Unlike a cookie, the
information is
    // stored on the server-side and cannot be modified by the visitor.  However,
    // note that in most cases sessions do still use cookies and require the visitor
    // to have cookies enabled.  For more information about sessions:
    // http://us.php.net/manual/en/book.session.php
    session_start();

    // Note that it is a good practice to NOT end your PHP files with a closing PHP
tag.
    // This prevents trailing newlines on the file from being included in your
output,
    // which can cause problems with redirecting users.

?>
```

If you don't know a whole lot about PHP, don't worry, all we are doing is providing the credentials to access our database and then we are connecting to it. Once we have connected to our database, we can make queries which will change our database.

That about covers it for the basic setup of our local apache server, setting up our MySQL database, and testing some scripts. Soon, we will be creating our web service and developing out our Android application. In the next tutorial we will be going over the exact same thing but on an actual server, such as, a hosting plan from Host Gator or any other hosting provider. If you feel comfortable with the information in this tutorial, and don't have a hosting plan, feel free to skip the next tutorial and jump straight into developing our web service.

Previous Lesson Donate Next Lesson

Author: trav

I'm just an average guy that love programming.

Share This Post On

Submit a Comment

Your email address will not be published. Required fields are marked *

Name *

Email *

Website

CAPTCHA Image

CAPTCHA Code *

Comment

You may use these HTML tags and attributes: `<a href="" title=""> <abbr title=""> <acronym title=""> <b> <blockquote cite=""> <cite> <code> <del datetime=""> <em> <i> <q cite=""> <strike> <strong>`

Submit Comment

Tutorials